

Project: Il problema del bandit

esplorazione e sfruttamento, apprendimento sequenziale e decisioni
sotto incertezza

1. Obiettivi della dispensa

Questa dispensa introduce il problema del bandit multi-armed come caso di studio per un corso di metodi computazionali per modelli stocastici.

Gli obiettivi sono sei:

1. formalizzare il problema della decisione sequenziale sotto incertezza come processo stocastico;
2. definire il regret cumulato come misura della qualità di una strategia;
3. introdurre e confrontare tre famiglie di algoritmi: epsilon-greedy, UCB e Thompson sampling;
4. dimostrare come la stessa matematica descriva contesti apparentemente molto diversi — sperimentazione clinica e test A/B su piattaforme digitali;
5. discutere i limiti teorici del problema tramite il lower bound di Lai-Robbins;
6. collegare il bandit al più generale tema dell'exploration-exploitation, trasversale a molti altri modelli del corso.

Dal punto di vista del corso, questo progetto introduce la forma più pura del trade-off exploration-exploitation: un agente deve decidere ripetutamente quale azione compiere, raccogliendo informazioni sulle conseguenze solo delle azioni che sceglie, mai di quelle che non sceglie. Questa struttura — imparare agendo, senza poter osservare i controfattuali — è fondamentalmente diversa dall'ottimizzazione classica e collega il bandit al modello di March, al foraging, al simulated annealing e alle dinamiche replicative.

2. Motivazione: due contesti, una matematica

2.1 Sperimentazione clinica adattiva

Un ospedale sta testando k trattamenti per una malattia — per esempio $k = 4$ diverse terapie per una forma di leucemia refrattaria. I pazienti arrivano in sequenza, uno alla volta. Per ogni paziente si deve scegliere quale trattamento

somministrare, e dopo qualche settimana si osserva l'esito: risposta completa, risposta parziale, nessuna risposta.

Il problema ha due componenti in tensione.

Da un lato, per imparare quale trattamento e' il migliore bisogna raccogliere dati su tutti e quattro — anche su quelli che sembrano peggiori. Questo e' il lato dell'**esplorazione**: sperimentare trattamenti incerti per ridurre l'ignoranza.

Dall'altro, ogni paziente assegnato a un trattamento subottimale e' un paziente che non ha ricevuto il trattamento migliore disponibile. Questo e' il lato dello **sfruttamento**: usare la conoscenza accumulata per massimizzare i benefici immediati.

La tensione e' reale e ha conseguenze concrete: un protocollo troppo esplorativo alloca troppi pazienti a terapie inefficaci prima di convergere alla migliore; un protocollo troppo sfruttante converge troppo in fretta a una terapia che sembra buona ma potrebbe non esserlo.

I trial clinici adattativi — approvati dalla FDA e dall'EMA — usano esattamente la logica del bandit per bilanciare queste due esigenze, modificando le probabilita' di assegnazione nel corso del trial in base ai risultati osservati.

2.2 Test A/B su piattaforme digitali

Una piattaforma di e-commerce vuole ottimizzare il layout della pagina prodotto. Ha $k = 5$ varianti del layout, ciascuna con un tasso di conversione sconosciuto: la frazione di visitatori che effettivamente acquista.

Ogni visitatore che arriva al sito viene assegnato a una variante. Si osserva se acquista o no. L'obiettivo e' massimizzare le conversioni totali nel corso del tempo.

La struttura e' identica al caso clinico:

- ogni variante e' un “braccio” del bandit;
- il tasso di conversione e' la ricompensa media;
- esplorare significa mostrare varianti incerte a qualche visitatore (perdendo potenziali conversioni);
- sfruttare significa mostrare la variante che sembra migliore a tutti i visitatori (rischiando di perdere qualcosa se la stima e' sbagliata).

La differenza pratica e' nell'orizzonte e nelle stakes: nel caso clinico gli errori hanno conseguenze mediche dirette; nel caso digitale il ritmo e' piu' rapido, i dati arrivano in tempo reale e il numero di “pazienti” e' molto piu' grande. Ma il problema matematico e' lo stesso.

Mostrare che la stessa struttura formale descrive questi due contesti cosi' diversi e' uno degli obiettivi centrali della dispensa.

3. Definizione formale del problema

3.1 Il bandit multi-armed

Un bandit multi-armed (o multi-armed bandit, MAB) e' definito da:

- k bracci (azioni, trattamenti, varianti), indicizzati da $a \in \{1, \dots, k\}$;
- per ogni braccio a , una distribuzione di ricompensa P_a con media sconosciuta μ_a ;
- un orizzonte temporale T (numero totale di decisioni).

Al tempo $t = 1, 2, \dots, T$:

1. l'agente sceglie un braccio $A_t \in \{1, \dots, k\}$;
2. riceve una ricompensa $R_t \sim P_{A_t}$;
3. osserva R_t ma non le ricompense degli altri bracci.

L'agente non conosce le medie μ_a e le deve stimare dalle osservazioni passate. Il punto 3 e' cruciale: si osserva solo la ricompensa del braccio scelto, mai quella dei bracci non scelti. Questo e' il **problema del controfattuale**: non si sa cosa sarebbe successo scegliendo diversamente.

3.2 Il braccio ottimale

Il braccio ottimale e' quello con la media piu' alta:

$$a^* = \arg \max_a \mu_a, \quad \mu^* = \mu_{a^*}.$$

Un agente che conosce μ_a per ogni braccio sceglierebbe sempre a^* e otterrebbe ricompensa attesa μ^* a ogni passo.

3.3 Il regret

Il **regret cumulato** misura quanto l'agente perde rispetto all'agente onnisciente che conosce il braccio ottimale:

$$R_T = \sum_{t=1}^T (\mu^* - \mu_{A_t}) = \sum_{a=1}^k \Delta_a N_a(T),$$

dove:

- $\Delta_a = \mu^* - \mu_a \geq 0$ e' il **gap** del braccio a rispetto all'ottimo;
- $N_a(T) = \sum_{t=1}^T \mathbf{1}\{A_t = a\}$ e' il numero di volte che il braccio a e' stato scelto.

Il regret e' zero solo se l'agente sceglie sempre il braccio ottimale. In pratica, il regret cresce nel tempo perche' l'agente deve esplorare bracci subottimali per imparare.

Interpretazione clinica. Se μ_a e' il tasso di risposta del trattamento a , il regret e' il numero di pazienti che non hanno ricevuto il trattamento ottimale — una misura diretta del danno causato dall'incertezza.

Interpretazione digitale. Se μ_a e' il tasso di conversione della variante a , il regret e' il numero di conversioni perse rispetto a mostrare sempre la variante migliore.

3.4 Il lower bound di Lai-Robbins

Lai e Robbins (1985) hanno dimostrato che nessun algoritmo può fare meglio di

$$R_T \geq \sum_{a: \Delta_a > 0} \frac{\Delta_a}{\text{KL}(\mu_a, \mu^*)} \ln T + o(\ln T),$$

dove $\text{KL}(\mu_a, \mu^*)$ e' la divergenza di Kullback-Leibler tra le distribuzioni dei due bracci.

In termini semplici: il regret cresce almeno come $\Omega(\ln T)$. Non si può fare meglio di una crescita logaritmica nel tempo. Un algoritmo che raggiunge questa crescita e' detto **asintoticamente ottimale**.

Questo risultato fissa il limite teorico: qualsiasi algoritmo ragionevole esplorerà sempre almeno un numero logaritmico di volte i bracci subottimali.

4. Distribuzioni di ricompensa

4.1 Bracci Bernoulliani

Il caso più comune nelle applicazioni e' il braccio Bernoulliano: la ricompensa e' 0 o 1. Il parametro $\mu_a \in [0, 1]$ e' la probabilita' di successo.

Caso clinico: risposta al trattamento (1) o assenza di risposta (0).

Caso digitale: acquisto (1) o nessun acquisto (0).

4.2 Bracci gaussiani

In alcune applicazioni la ricompensa e' continua. Il braccio a ha distribuzione

$$R_t \sim \mathcal{N}(\mu_a, \sigma^2),$$

con σ^2 noto o stimato. Questo e' più comune in applicazioni di pricing o in contesti sperimentali in cui la ricompensa e' una misura quantitativa.

4.3 Stima empirica

Dopo n_a osservazioni del braccio a , la stima naturale della media e':

$$\hat{\mu}_a = \frac{1}{n_a} \sum_{t:A_t=a} R_t.$$

Per i bracci Bernoulliani questa e' la frequenza di successi. Per i bracci gaussiani e' la media campionaria.

5. Algoritmi

5.1 Epsilon-greedy

L'epsilon-greedy e' l'algoritmo piu' semplice. Ad ogni passo:

- con probabilita' $1 - \varepsilon$: sceglie il braccio con la media stimata piu' alta (**sfruttamento**);
- con probabilita' ε : sceglie un braccio casuale (**esplorazione**).

$$A_t = \begin{cases} \arg \max_a \hat{\mu}_a & \text{con probabilita' } 1 - \varepsilon, \\ \text{braccio casuale} & \text{con probabilita' } \varepsilon. \end{cases}$$

Variante decrescente. Si puo' far decrescere ε nel tempo, ad esempio $\varepsilon_t = \varepsilon_0/t$. Così' si esplora molto all'inizio e si sfrutta di piu' verso la fine. Con $\varepsilon_t \propto 1/t$ il regret cresce come $O(\ln T)$.

Limite dell'epsilon-greedy. Esplora in modo cieco, senza tener conto di quanto si e' incerti su ogni braccio. Un braccio gia' stimato con precisione riceve la stessa attenzione esplorativa di uno mai provato.

5.2 UCB (Upper Confidence Bound)

L'algoritmo UCB (Auer et al., 2002) usa l'incertezza sulla stima come criterio di esplorazione: sceglie il braccio con il valore ottimisticamente piu' alto.

Alla iterazione t , si calcola per ogni braccio a :

$$\text{UCB}_a(t) = \hat{\mu}_a + \sqrt{\frac{2 \ln t}{N_a(t)}}.$$

Si sceglie $A_t = \arg \max_a \text{UCB}_a(t)$.

Interpretazione. Il termine $\hat{\mu}_a$ e' la stima corrente. Il termine $\sqrt{2 \ln t / N_a(t)}$ e' un **bonus di esplorazione**: e' grande quando $N_a(t)$ e' piccolo (braccio poco esplorato) e decresce quando $N_a(t)$ cresce (stima piu' affidabile). L'algoritmo e'

ottimista: tratta ogni braccio come se potesse essere buono quanto suggerisce l'intervallo di confidenza superiore.

Proprieta' teoriche. UCB raggiunge un regret $O(\ln T)$, vicino al limite di Lai-Robbins. Non richiede di specificare ε o altri iperparametri difficili da tarare.

Caso clinico. UCB esplora sistematicamente i trattamenti poco testati, indipendentemente da quanto sembrano promettenti. Questo garantisce che nessun trattamento venga abbandonato prematuramente per sfortuna statistica nelle prime osservazioni.

5.3 Thompson sampling

Il Thompson sampling (Thompson, 1933; riscoperto negli anni 2000) e' un approccio bayesiano. Mantiene una distribuzione a posteriori su μ_a per ogni braccio e campiona da essa per decidere quale braccio scegliere.

Schema per bracci Bernoulliani con prior Beta.

Si usa il prior coniugato $\mu_a \sim \text{Beta}(\alpha_a, \beta_a)$.

Inizialmente: $\alpha_a = \beta_a = 1$ (prior uniforme su $[0, 1]$) per ogni braccio.

Ad ogni passo t :

1. per ogni braccio a , campiona $\theta_a \sim \text{Beta}(\alpha_a, \beta_a)$;
2. scegli $A_t = \arg \max_a \theta_a$;
3. osserva R_t ; aggiorna: se $R_t = 1$ poni $\alpha_{A_t} \leftarrow \alpha_{A_t} + 1$, se $R_t = 0$ poni $\beta_{A_t} \leftarrow \beta_{A_t} + 1$.

Interpretazione. A ogni passo si estrae un valore plausibile di μ_a dalla distribuzione a posteriori di ogni braccio. Si sceglie il braccio con il valore estratto piu' alto. L'esplorazione emerge naturalmente dall'incertezza: i bracci poco testati hanno distribuzioni a posteriori molto larghe, quindi spesso producono campioni estremi che li fanno scegliere.

Aggiornamento bayesiano. La distribuzione Beta e' il prior coniugato per la Bernoulliana: se $\mu_a \sim \text{Beta}(\alpha, \beta)$ e si osservano s successi e f fallimenti, la posteriore e' $\text{Beta}(\alpha + s, \beta + f)$. L'aggiornamento e' quindi semplicissimo: si contano successi e fallimenti.

Caso clinico. Thompson sampling ha una giustificazione bayesiana molto naturale in ambito medico: si esplora in proporzione all'incertezza, e si sfrutta in proporzione alla probabilita' che un trattamento sia il migliore.

Proprieta' teoriche. Thompson sampling raggiunge un regret $O(\ln T)$, spesso con costanti migliori di UCB in pratica.

5.4 Confronto tra algoritmi

Algoritmo	Esplorazione	Parametri	Limite teorico
Epsilon-greedy	casuale, quota fissa	ε	$O(T)$ fisso, $O(\ln T)$ decrescente
UCB	ottimistica, basata su confidenza	nessuno	$O(\ln T)$
Thompson sampling	bayesiana, basata su incertezza	prior	$O(\ln T)$

6. Il regret come processo stocastico

6.1 Traiettorie del regret

Il regret cumulato R_T e' una variabile aleatoria. La sua traiettoria nel tempo mostra:

- salti quando si sceglie un braccio subottimale;
- periodi piatti quando si sceglie il braccio ottimale.

Un algoritmo di buona qualita' produce traiettorie che crescono sempre piu' lentamente, perche' converge progressivamente al braccio ottimale.

6.2 Convergenza delle stime

Una osservabile utile e' l'evoluzione di $\hat{\mu}_a(t)$ nel tempo. Per il braccio ottimale, la stima deve convergere verso μ^* con fluttuazioni decrescenti. Per i bracci subottimali, le stime possono essere meno precise (vengono esplorati meno).

6.3 Frequenza di selezione

Un'altra osservabile e' la frazione di volte che ogni braccio viene scelto:

$$f_a(T) = \frac{N_a(T)}{T}.$$

Un buon algoritmo dovrebbe avere $f_{a^*}(T) \rightarrow 1$ per $T \rightarrow \infty$: la quasi totalita' delle scelte converge verso il braccio ottimale, con gli altri bracci esplorati un numero logaritmico di volte.

7. I due esempi a confronto

7.1 Struttura comune

La tabella seguente mostra come i due contesti condividano la stessa struttura matematica.

Elemento	Sperimentazione clinica	Test A/B digitale
Braccio a	trattamento	variante del layout
Ricompensa R_t	risposta terapeutica (0/1)	conversione (0/1)
μ_a	tasso di risposta	tasso di conversione
T	numero di pazienti arruolati	numero di visitatori
Regret	pazienti non trattati ottimalmente	conversioni perse
Esplorazione	assegnare pazienti a terapie incerte	mostrare varianti incerte ai visitatori
Sfruttamento	assegnare pazienti alla terapia migliore nota	mostrare la variante migliore nota

7.2 Differenze pratiche

Nonostante la struttura matematica identica, i due contesti hanno differenze importanti che influenzano la scelta dell'algoritmo e dei parametri.

Ritardo delle osservazioni. In un trial clinico la risposta al trattamento può essere osservata settimane o mesi dopo l'assegnazione. In un test A/B digitale la conversione è quasi istantanea. Il ritardo nel feedback modifica l'algoritmo: con osservazioni ritardate si deve decidere chi assegnare a quale braccio prima di avere tutti i risultati in mano.

Stakes. In ambito clinico un errore sistematico ha conseguenze mediche dirette. In ambito digitale le conseguenze sono economiche. Questo influenza quanto si è disposti ad accettare regret per ridurre il rischio di errori.

Orizzonti. Un trial clinico ha tipicamente T dell'ordine di centinaia o migliaia di pazienti. Un test A/B su una grande piattaforma ha T dell'ordine di milioni di visitatori. Con T molto grande, anche un piccolo miglioramento dell'algoritmo produce grandi guadagni assoluti.

Vincoli etici e regolatori. In ambito clinico, i protocolli di sperimentazione devono essere approvati da comitati etici e rispettare normative precise. Non si può semplicemente cambiare la probabilità di assegnazione in tempo reale senza giustificazione. In ambito digitale la flessibilità è molto maggiore.

7.3 Il messaggio metodologico

Il punto non è che i due contesti siano identici: le differenze pratiche sono reali e importanti. Il punto è che la stessa struttura matematica — bracci, ricompense, regret, algoritmi di bilanciamento esplorazione/sfruttamento — si applica a entrambi, e che imparare a risolvere il problema in un contesto fornisce strumenti immediatamente trasferibili all'altro.

Questo e' esattamente il tipo di trasferibilita' che rende utile un corso di metodi computazionali per modelli stocastici.

8. Connessioni con altri modelli del corso

8.1 Modello di March

Il modello di March studia il trade-off esplorazione/sfruttamento nelle organizzazioni: investire in nuove pratiche (esplorazione) vs consolidare quelle esistenti (sfruttamento). Il bandit formalizza questa tensione in modo quantitativo: il regret misura il costo dell'esplorazione, e gli algoritmi mostrano come bilanciare le due esigenze in modo ottimale.

8.2 Foraging ottimale

Il problema del foraging — quanto tempo restare in una patch prima di spostarsi — ha la stessa struttura del bandit: ogni patch e' un braccio, il guadagno per unita' di tempo e' la ricompensa, e il forager deve bilanciare lo sfruttamento della patch corrente con l'esplorazione di patch nuove. Il teorema del valore marginale e' la soluzione ottimale per un caso deterministico; gli algoritmi bandit generalizzano al caso stocastico.

8.3 Dinamiche replicative

La selezione proporzionale alla fitness nelle dinamiche replicative e' formalmente analoga alla selezione in Thompson sampling: le strategie con payoff alto aumentano di frequenza, quelle con payoff basso diminuiscono. La differenza e' che nelle dinamiche replicative l'aggiornamento e' collettivo e deterministico nel limite di grande popolazione, mentre in Thompson sampling e' individuale e stocastico.

8.4 Simulated annealing e SGD

In SA, la temperatura alta corrisponde a molta esplorazione dello spazio delle soluzioni; la temperatura bassa a molto sfruttamento della soluzione corrente. In SGD, il learning rate alto esplora, quello basso sfrutta. Il parametro ϵ nell'epsilon-greedy ha la stessa funzione.

9. Domande scientifiche che il modello permette di studiare

Il modello e' utile per affrontare domande precise.

1. Come cambia il regret cumulato al variare del numero di bracci k e dell'orizzonte T ?

2. Quanto conta la differenza tra i gap Δ_a ? Un problema con gap piccoli e' piu' difficile di uno con gap grandi?
3. UCB e Thompson sampling raggiungono entrambi $O(\ln T)$: differiscono nelle costanti? Quale e' migliore in pratica?
4. Come si comporta epsilon-greedy a epsilon fisso vs epsilon decrescente?
5. Quante osservazioni servono prima che la stima del braccio ottimale sia affidabile?
6. Come cambia la performance degli algoritmi al variare della varianza delle ricompense?

10. Schema del laboratorio

10.1 Laboratorio 1 - Il problema di base e il regret

Obiettivo

Implementare il bandit Bernoulliano con k bracci e confrontare le tre strategie.

Attività

1. definire $k = 5$ bracci con medie μ_a fissate (ad esempio $\mu = [0.3, 0.5, 0.7, 0.4, 0.6]$);
2. implementare epsilon-greedy, UCB e Thompson sampling;
3. simulare $T = 1000$ passi per ogni algoritmo;
4. confrontare le traiettorie del regret cumulato e la frequenza di selezione di ogni braccio.

Domande guida

- quale algoritmo converge piu' velocemente al braccio ottimale?
- UCB esplora in modo diverso da Thompson sampling?
- il regret di epsilon-greedy a ϵ fisso continua a crescere linearmente?

Output richiesto

- codice sorgente;
- grafici del regret cumulato per i tre algoritmi;
- grafici della frequenza di selezione di ogni braccio nel tempo;
- commento interpretativo.

10.2 Laboratorio 2 - Il ruolo dei gap

Obiettivo

Studiare come la difficoltà del problema dipende dalla struttura dei gap Δ_a .

Attivita'

1. confrontare due scenari con lo stesso braccio ottimale ($\mu^* = 0.7$) ma gap diversi:
 - scenario facile: $\mu = [0.1, 0.2, 0.3, 0.4, 0.7]$;
 - scenario difficile: $\mu = [0.6, 0.62, 0.64, 0.66, 0.7]$;
2. per ogni scenario, eseguire 100 run indipendenti;
3. confrontare la distribuzione del regret a $T = 2000$.

Domande guida

- il regret e' piu' alto nello scenario difficile o in quello facile?
- gli algoritmi si comportano diversamente nei due scenari?
- il lower bound di Lai-Robbins e' visibile nei risultati?

Output richiesto

- boxplot del regret finale per i due scenari;
- confronto delle curve di regret medie;
- commento sulla relazione tra gap e difficolta'.

10.3 Laboratorio 3 - Sperimentazione clinica adattiva

Obiettivo

Simulare un trial clinico adattivo con $k = 4$ trattamenti e confrontare il protocollo bandit con un protocollo a randomizzazione uniforme.

Attivita'

1. definire $k = 4$ trattamenti con tassi di risposta $\mu = [0.2, 0.4, 0.6, 0.35]$;
2. simulare il trial con Thompson sampling per $T = 200$ pazienti;
3. simulare lo stesso trial con randomizzazione uniforme (ogni trattamento con probabilita' $1/4$);
4. confrontare il numero di pazienti assegnati al trattamento ottimale e il regret totale.

Domande guida

- quanti pazienti in piu' ricevono il trattamento ottimale con Thompson sampling rispetto alla randomizzazione uniforme?
- a quale punto del trial il protocollo adattivo converge chiaramente alla terapia migliore?
- il protocollo adattivo rischia di convergere prematuramente a un trattamento subottimale?

Output richiesto

- grafici dell’allocazione dei pazienti nel tempo per ogni trattamento;
- confronto del regret tra bandit e randomizzazione uniforme;
- stima della “vita media” del trial (quando la terapia ottimale diventa chiaramente dominante).

10.4 Laboratorio 4 - Test A/B adattivo

Obiettivo

Simulare un test A/B adattivo su una piattaforma digitale con $k = 3$ varianti e T grande.

Attività

1. definire $k = 3$ varianti con tassi di conversione $\mu = [0.05, 0.08, 0.06]$;
2. simulare con UCB per $T = 10000$ visitatori;
3. confrontare con un test A/B classico a due fasi: prima $T/2$ visitatori distribuiti uniformemente, poi i restanti tutti sulla variante migliore stimata;
4. confrontare il numero totale di conversioni e il regret.

Domande guida

- il test A/B classico a due fasi è efficiente?
- con T grande, UCB produce molte più conversioni?
- come cambia il confronto per $T = 1000$ vs $T = 100000$?

Output richiesto

- confronto delle conversioni totali tra UCB e test A/B classico;
- grafici della frequenza di selezione delle varianti nel tempo;
- commento su quando vale la pena usare un approccio adattivo.

11. Una possibile estensione teorica

11.1 Il bandit contestuale

Nel bandit classico le distribuzioni dei bracci sono fisse. Nel **bandit contestuale**, prima di ogni decisione si osserva un contesto x_t (caratteristiche del paziente, o del visitatore web) che contiene informazioni sulla ricompensa attesa di ogni braccio.

Formalmente, la ricompensa attesa dipende sia dal braccio sia dal contesto:

$$\mathbb{E}[R_t \mid A_t = a, x_t] = f(a, x_t).$$

Questo e' molto piu' realistico: in ambito clinico le caratteristiche del paziente (eta', stadio della malattia, biomarcatori) influenzano la risposta al trattamento. In ambito digitale il comportamento di ogni visitatore dipende da dove viene, da cosa ha gia' visto, dall'ora del giorno.

Il bandit contestuale introduce una nuova sfida: bisogna imparare non solo quale braccio e' mediamente il migliore, ma come la risposta di ogni braccio dipende dal contesto.

11.2 Connessione con l'apprendimento per rinforzo

Il bandit e' il caso piu' semplice dell'apprendimento per rinforzo: c'e' un solo stato, e ogni azione produce immediatamente una ricompensa. L'apprendimento per rinforzo generalizza questo schema a problemi con piu' stati e ricompense ritardate. Gli algoritmi sviluppati per il bandit (in particolare UCB e Thompson sampling) hanno ispirato molti metodi dell'apprendimento per rinforzo moderno.

12. Perche' questo e' un buon case study per il corso

Questo progetto e' particolarmente adatto a un corso di metodi computazionali per modelli stocastici per almeno cinque ragioni.

Primo, introduce la forma piu' pura del trade-off exploration-exploitation, che e' un tema trasversale a molti altri modelli del corso (March, foraging, SA, SGD).

Secondo, il regret e' una misura di performance molto concreta: ha un'interpretazione immediata sia in termini medici (pazienti non trattati ottimalmente) sia in termini economici (conversioni perse).

Terzo, i due esempi applicativi mostrano come la stessa struttura matematica si trasferisca in modo diretto tra domini molto diversi, il che e' uno degli obiettivi metodologici del corso.

Quarto, gli algoritmi sono semplici da implementare ma con proprieta' teoriche non banali: epsilon-greedy e' una riga di codice ma ha un regret lineare; UCB ha un regret logaritmico garantito con pochissimi iperparametri.

Quinto, Thompson sampling introduce il campionamento da distribuzioni a posteriori (Beta) in un contesto molto concreto, collegando il progetto alla lezione su Monte Carlo e MCMC.

13. Conclusione

Il problema del bandit mostra come una situazione di apprendimento sequenziale — decidere quale azione compiere, raccogliere informazioni, aggiornare le stime,

decidere di nuovo — possa essere formalizzata in modo preciso e risolta con algoritmi con garanzie teoriche.

Il messaggio concettuale piu' importante e' che **l'esplorazione ha un costo, ma non esplorare ha un costo ancora maggiore**. Un agente che sfrutta sempre la migliore alternativa nota non impara mai se ci sia qualcosa di meglio. Un agente che esplora troppo paga un prezzo inutile. Gli algoritmi ottimali bilanciano esattamente queste due esigenze, esplorando in modo decrescente nel tempo e convergendo progressivamente all'alternativa migliore.

Dal punto di vista metodologico, il progetto combina in modo naturale:

- definizione rigorosa di un problema di decisione sotto incertezza;
- misura di performance quantitativa (regret);
- tre famiglie algoritmiche con diverse filosofie (frequentista, ottimistica, bayesiana);
- due contesti applicativi con la stessa struttura matematica;
- connessioni con altri modelli del corso.

14. Bibliografia minima

1. Robbins, H. (1952). Some Aspects of the Sequential Design of Experiments. *Bulletin of the American Mathematical Society*, 58(5), 527-535.
2. Lai, T. L., and Robbins, H. (1985). Asymptotically Efficient Adaptive Allocation Rules. *Advances in Applied Mathematics*, 6(1), 4-22.
3. Auer, P., Cesa-Bianchi, N., and Fischer, P. (2002). Finite-time Analysis of the Multiarmed Bandit Problem. *Machine Learning*, 47(2-3), 235-256.
4. Thompson, W. R. (1933). On the Likelihood that One Unknown Probability Exceeds Another in View of the Evidence of Two Samples. *Biometrika*, 25(3-4), 285-294.
5. Lattimore, T., and Szepesvari, C. (2020). *Bandit Algorithms*. Cambridge University Press.

Appendice A. Implementazione in Python: struttura del codice

Questa appendice propone una struttura semplice per implementare in Python i tre algoritmi principali e le analisi del progetto.

A.1 Librerie minime

```
import random
import math
```

```
import statistics
import matplotlib.pyplot as plt
```

A.2 Struttura del bandit

```
def sample_reward(mu):
    return 1 if random.random() < mu else 0

def best_arm(mus):
    return max(range(len(mus)), key=lambda a: mus[a])
```

A.3 Epsilon-greedy

```
def epsilon_greedy(mus, T, epsilon):
    k = len(mus)
    counts = [0] * k
    estimates = [0.0] * k
    regret = []
    mu_star = max(mus)
    cumulative_regret = 0.0

    for t in range(T):
        if random.random() < epsilon:
            arm = random.randint(0, k - 1)
        else:
            arm = max(range(k), key=lambda a: estimates[a])

        reward = sample_reward(mus[arm])
        counts[arm] += 1
        estimates[arm] += (reward - estimates[arm]) / counts[arm]

        cumulative_regret += mu_star - mus[arm]
        regret.append(cumulative_regret)

    return regret, counts, estimates
```

L'aggiornamento incrementale `estimates[arm] += (reward - estimates[arm]) / counts[arm]` e' equivalente alla media campionaria ma non richiede di memorizzare tutte le ricompense passate.

A.4 UCB

```
def ucb(mus, T):
    k = len(mus)
    counts = [0] * k
    estimates = [0.0] * k
```

```

regret = []
mu_star = max(mus)
cumulative_regret = 0.0

# inizializza: prova ogni braccio una volta
for arm in range(k):
    reward = sample_reward(mus[arm])
    counts[arm] = 1
    estimates[arm] = reward
    cumulative_regret += mu_star - mus[arm]
    regret.append(cumulative_regret)

for t in range(k, T):
    ucb_values = [
        estimates[a] + math.sqrt(2.0 * math.log(t + 1) / counts[a])
        for a in range(k)
    ]
    arm = max(range(k), key=lambda a: ucb_values[a])

    reward = sample_reward(mus[arm])
    counts[arm] += 1
    estimates[arm] += (reward - estimates[arm]) / counts[arm]

    cumulative_regret += mu_star - mus[arm]
    regret.append(cumulative_regret)

return regret, counts, estimates

```

A.5 Thompson sampling con prior Beta

```

def thompson_sampling(mus, T):
    k = len(mus)
    alpha = [1.0] * k
    beta_params = [1.0] * k
    regret = []
    mu_star = max(mus)
    cumulative_regret = 0.0

    for t in range(T):
        # campiona da ogni distribuzione a posteriori
        samples = [
            random.betavariate(alpha[a], beta_params[a])
            for a in range(k)
        ]
        arm = max(range(k), key=lambda a: samples[a])

```

```

reward = sample_reward(mus[arm])

# aggiorna la posteriore
if reward == 1:
    alpha[arm] += 1.0
else:
    beta_params[arm] += 1.0

cumulative_regret += mu_star - mus[arm]
regret.append(cumulative_regret)

return regret, alpha, beta_params

```

A.6 Confronto tra algoritmi su molte run

```

def run_many(algorithm_func, mus, T, n_runs, **kwargs):
    all_regrets = []

    for run in range(n_runs):
        regret, _, _ = algorithm_func(mus, T, **kwargs)
        all_regrets.append(regret)

    mean_regret = [
        statistics.mean(all_regrets[run][t] for run in range(n_runs))
        for t in range(T)
    ]

    return mean_regret

def plot_regret_comparison(mus, T, n_runs=100, epsilon=0.1):
    regret_eg = run_many(epsilon_greedy, mus, T, n_runs, epsilon=epsilon)
    regret_ucb = run_many(ucb, mus, T, n_runs)
    regret_ts = run_many(thompson_sampling, mus, T, n_runs)

    times = list(range(T))

    plt.plot(times, regret_eg, label=f"epsilon-greedy (eps={epsilon})")
    plt.plot(times, regret_ucb, label="UCB")
    plt.plot(times, regret_ts, label="Thompson sampling")
    plt.xlabel("tempo t")
    plt.ylabel("regret cumulato medio")
    plt.title("Confronto tra algoritmi bandit")
    plt.legend()
    plt.show()

```

A.7 Frequenza di selezione dei bracci

```
def plot_arm_selection(counts, mus, title="Frequenza di selezione"):
    k = len(mus)
    T = sum(counts)
    fractions = [counts[a] / T for a in range(k)]
    labels = [f"braccio {a+1} ( $\mu$ ={mus[a]})" for a in range(k)]

    plt.bar(range(k), fractions)
    plt.xticks(range(k), labels, rotation=15)
    plt.ylabel("frazione delle selezioni")
    plt.title(title)
    plt.show()
```

A.8 Simulazione del trial clinico

```
def clinical_trial_comparison(mus, T, n_runs=200):
    regret_ts = run_many(thompson_sampling, mus, T, n_runs)

    # randomizzazione uniforme
    all_regrets_uniform = []
    k = len(mus)
    mu_star = max(mus)

    for run in range(n_runs):
        cumulative_regret = 0.0
        regret = []
        for t in range(T):
            arm = random.randint(0, k - 1)
            cumulative_regret += mu_star - mus[arm]
            regret.append(cumulative_regret)
        all_regrets_uniform.append(regret)

    mean_regret_uniform = [
        statistics.mean(all_regrets_uniform[run][t] for run in range(n_runs))
        for t in range(T)
    ]

    times = list(range(T))
    plt.plot(times, regret_ts, label="Thompson sampling")
    plt.plot(times, mean_regret_uniform, label="randomizzazione uniforme")
    plt.xlabel("paziente t")
    plt.ylabel("regret cumulato medio")
    plt.title("Trial clinico: bandit vs randomizzazione uniforme")
    plt.legend()
    plt.show()
```

```

print("Regret finale Thompson sampling:", round(regret_ts[-1], 1))
print("Regret finale randomizzazione uniforme:",
      round(mean_regret_uniform[-1], 1))

```

A.9 Distribuzione a posteriori nel tempo

Visualizzare come le distribuzioni Beta evolvono nel tempo e' utile per capire Thompson sampling:

```

def plot_posterior_evolution(mus, T_snapshots, T_total=1000):
    k = len(mus)
    alpha = [1.0] * k
    beta_params = [1.0] * k
    mu_star = max(mus)

    for t in range(T_total):
        samples = [random.betavariate(alpha[a], beta_params[a]) for a in range(k)]
        arm = max(range(k), key=lambda a: samples[a])
        reward = sample_reward(mus[arm])

        if reward == 1:
            alpha[arm] += 1.0
        else:
            beta_params[arm] += 1.0

        if t + 1 in T_snapshots:
            print(f"\nDopo t={t+1}:")
            for a in range(k):
                mean_post = alpha[a] / (alpha[a] + beta_params[a])
                print(f" braccio {a+1}: alpha={alpha[a]:.0f}, "
                      f"beta={beta_params[a]:.0f}, "
                      f"media posteriore={mean_post:.3f} "
                      f"(vera={mus[a]})")

```

A.10 Esempio completo

```

if __name__ == "__main__":
    mus = [0.3, 0.5, 0.7, 0.4, 0.6]
    T = 1000
    n_runs = 200

    print("=== Confronto algoritmi ===")
    plot_regret_comparison(mus, T, n_runs=n_runs, epsilon=0.1)

    print("\n=== UCB: frequenza di selezione ===")

```

```

regret_ucb, counts_ucb, estimates_ucb = ucb(mus, T)
plot_arm_selection(counts_ucb, mus, title="UCB: frequenza di selezione")

print("\n=== Trial clinico ===")
mus_clinical = [0.2, 0.4, 0.6, 0.35]
clinical_trial_comparison(mus_clinical, T=200, n_runs=500)

print("\n=== Evoluzione delle posteriori (Thompson sampling) ===")
plot_posterior_evolution(mus, T_snapshots=[10, 50, 200, 1000])

```

A.11 Conclusione dell'appendice

L'implementazione proposta e' volutamente semplice. I tre algoritmi principali si scrivono ciascuno in una ventina di righe. Il punto metodologico e' che la complessita' del problema non sta nel codice ma nella struttura matematica: il regret, il lower bound di Lai-Robbins, e il trade-off tra esplorazione e sfruttamento sono concetti non banali che emergono anche da implementazioni molto elementari.